

Amazon Food Reviews

Sentiment Analysis Project

Exploratory Data Analysis, Feature Engineering & Machine Learning

Dataset: Amazon Fine Food Reviews (Kaggle)

Tools: Python | scikit-learn | NLTK | TensorFlow | XGBoost | Gensim

Task Type: Multi-Class Text Classification (Positive / Neutral / Negative)

Warnakulasuriyage Nilantha Chamara
Warnakulasuriya
Batch - Data Science 20/22

Contact Infor:

E-mail : nilanthaous@hotmail.com

Mobile: +16479287651

1. Executive Summary

Online reviews play a major role in shaping customer decisions, especially on large e-commerce platforms like Amazon. However, manually reading thousands of customer comments is both time-consuming and impractical. Because of this, the goal of this project was to build a machine learning system capable of automatically identifying customer sentiment from Amazon food product reviews.

The project focused on classifying reviews into three sentiment categories:

- Positive
- Neutral
- Negative

A working sample of 50,000 reviews was taken from the Amazon Fine Food Reviews dataset available on Kaggle. From there, the project moved through a complete NLP and machine learning workflow, beginning with exploratory data analysis and text preprocessing before progressing into feature engineering, model development, evaluation, and optimization.

Several machine learning approaches were explored during experimentation. Traditional models such as Logistic Regression, Naive Bayes, and Random Forest were tested alongside more advanced techniques including XGBoost and an LSTM neural network.

Different text representation methods were also compared, mainly:

- Bag of Words (BoW)
- TF-IDF
- Word2Vec embeddings

Among all tested combinations, TF-IDF with Logistic Regression delivered the strongest overall performance, achieving approximately 86.7% accuracy. Although Word2Vec produced semantically richer embeddings, it did not outperform TF-IDF on this dataset size.

One important observation was that neutral reviews proved much harder to classify than clearly positive or negative reviews. Many 3-star reviews contained mixed opinions, which naturally increased classification difficulty.

To make the system reusable in real-world applications, the final preprocessing pipeline, vectorizer, and trained model were serialized using joblib. This allows the prediction workflow to be integrated into future applications or deployed in production environments.

Key findings:

- TF-IDF with Logistic Regression delivered the best overall performance at approximately 86.68% accuracy.
- Word2Vec, while semantically richer, underperformed (83.97%) on this dataset size.
- Hyperparameter tuning via GridSearchCV yielded further marginal improvements.
- The final inference pipeline was serialized with joblib for production deployment.

2. Introduction

2.1 Background

Customer reviews are one of the most valuable sources of product feedback in modern e-commerce. Every day, online marketplaces generate massive amounts of text-based reviews containing customer opinions, complaints, preferences, and recommendations.

For companies, manually analyzing this information is almost impossible at scale. That is where Natural Language Processing (NLP) and machine learning become useful. These technologies make it possible to automatically identify sentiment patterns from large collections of unstructured text.

In this project, Amazon food reviews were used to build a sentiment classification system capable of predicting whether a review expresses positive, neutral, or negative sentiment.

2.2 Problem Statement

The main objective of the project was to predict customer sentiment using review text. The classification rules were defined using Amazon star ratings:

- Ratings of 4 or 5 → Positive
- Rating of 3 → Neutral
- Ratings of 1 or 2 → Negative

This created a multi-class text classification problem.

2.3 Objectives

The primary goal of this study is to design a robust pipeline that can ingest these raw reviews, perform mandatory text preprocessing (lowercasing, tokenization, and lemmatization), and accurately classify the customer's intent as Positive or Negative. This project emphasizes not only model accuracy but also the implementation of a reusable Inference Pipeline that can be scaled for production use.

The project aimed to:

1. Explore and understand the dataset using EDA techniques
2. Clean and preprocess customer review text
3. Create multiple feature engineering approaches
4. Train and compare several machine learning models
5. Optimize model performance using hyperparameter tuning
6. Build a reusable sentiment prediction pipeline

3. Dataset Description

3.1 Source

The dataset used in this project was the Amazon Fine Food Reviews dataset obtained from Kaggle. The data was downloaded programmatically using the opendatasets library.

The dataset contains customer reviews, review scores, and additional metadata related to Amazon food products.

The dataset sourced Link is (<https://www.kaggle.com/datasets/satyabrat35/amazon-food-review-dataset>).

3.2 Dataset Characteristics

Attribute	Details
Source	Kaggle — Amazon Fine Food Reviews
Working Sample Size	50,000 reviews (random_state=42)
Key Columns Used	Text (review body), Score (star rating 1–5)
Target Variable	Sentiment (Positive / Neutral / Negative)
Task Type	Multi-class text classification
Language	English

3.3 Sentiment Labeling

To convert numerical ratings into sentiment categories, the following mapping strategy was used:

Star Rating	Sentiment Label	Interpretation
4 or 5 stars	Positive	Satisfied customer
3 stars	Neutral	Mixed opinion
1 or 2 stars	Negative	Dissatisfied customer

This labeling approach helped simplify the sentiment prediction task while still preserving meaningful customer opinion categories.

3.4 Class Distribution

One clear pattern in the dataset was the strong dominance of positive reviews. This is very common in e-commerce datasets because customers often leave reviews after positive purchasing experiences.

Negative and neutral reviews appeared much less frequently, which introduced a moderate class imbalance problem.

This imbalance became important during evaluation because a model could achieve deceptively high accuracy simply by predicting the majority class most of the time.

.

4. Exploratory Data Analysis (EDA)

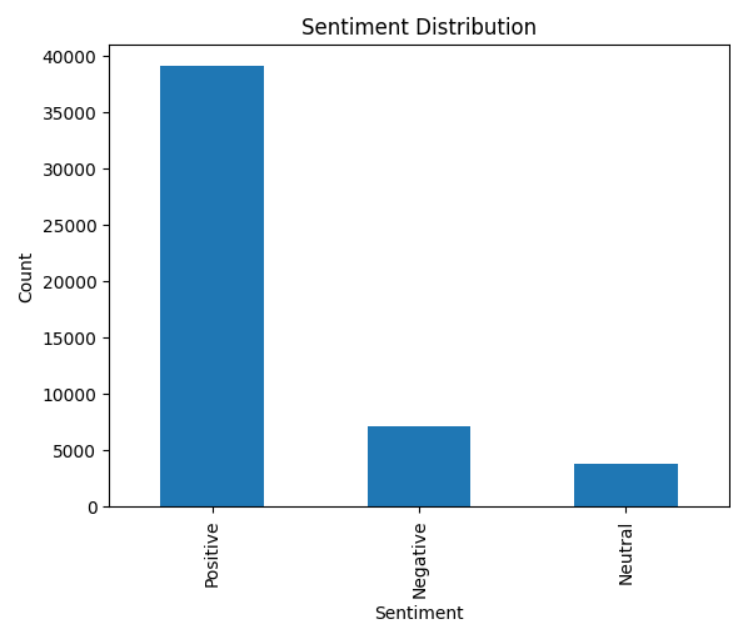
4.1 Data Quality Assessment

The following data quality checks were performed upon loading the dataset:

Check	Method	Finding
Missing Values	df.isnull().sum()	Minimal nulls; handled via fillna("")
Duplicate Rows	df.duplicated().sum()	Duplicates identified and flagged
Dataset Shape	df.shape	Full dataset inspected before sampling
Data Types	df.info()	Text (object), Score (int64) columns verified

Minimal missing values were found and handled using fillna(""). Duplicate reviews were also identified and removed where necessary.

4.2 Sentiment Distribution Analysis



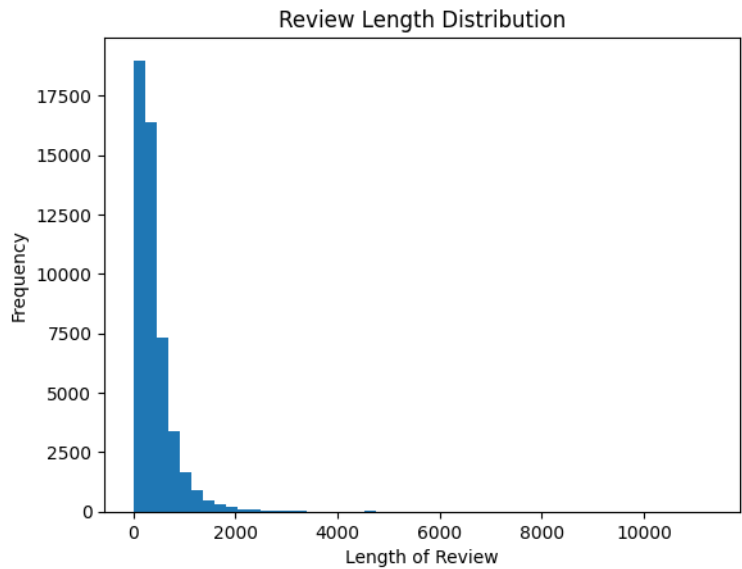
After generating sentiment labels, the class distribution was visualized using bar charts.

The results clearly showed that positive reviews dominated the dataset. Neutral and negative reviews represented much smaller portions of the data.

This imbalance highlighted the importance of evaluating models using metrics beyond simple accuracy.

4.3 Review Length Distribution

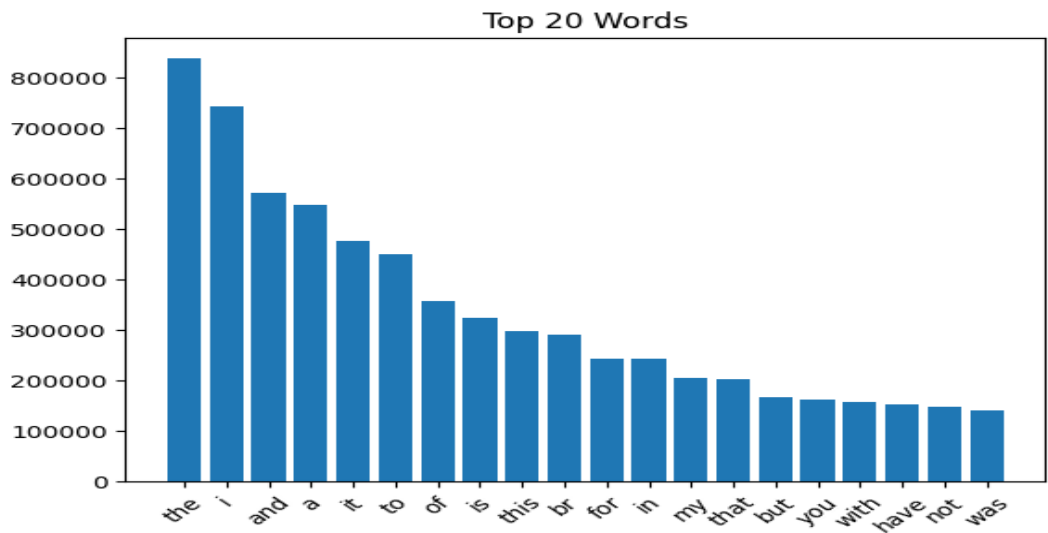
A histogram of review character lengths was generated. Key observations:



- Most reviews are relatively short (concentrated in the 0–500 character range).
 - A long tail of very lengthy reviews exists.
 - Short reviews may carry limited sentiment signal, potentially affecting model accuracy.
- Short reviews sometimes lacked strong contextual information, which may have reduced

classification quality for certain samples.

4.4 Word Frequency Analysis



Word frequency analysis was performed before preprocessing.

As expected, common stopwords such as:

- The / and / is / a and etc.....

appeared very frequently.

This confirmed the importance of stopwords removal during preprocessing.

After removing stopwords, frequently occurring meaningful words included:

- Good / love / great / taste / product etc.....

These terms reinforced the positive nature of the dataset.

4.5 EDA Summary

The exploratory data analysis reveals that the dataset is moderately imbalanced, with positive reviews dominating. The review length distribution shows that most reviews are relatively short, which may limit the depth of sentiment information. Word frequency analysis indicates that commonly used words are largely positive, suggesting an overall positive sentiment trend in the dataset. These findings highlight the need for preprocessing techniques such as balancing and feature engineering before model development.

5. Text Preprocessing

5.1 Preprocessing Pipeline

Raw customer reviews contained punctuation, stopwords, inconsistent capitalization, and other noisy elements that could reduce model quality.

To improve consistency, a complete preprocessing pipeline was implemented.

The pipeline included the following steps:

Step	Operation	Purpose
1	Lowercasing	Normalize case (e.g., 'Good' = 'good')
2	Remove Punctuation & Numbers	Eliminate non-alphabetic noise
3	Tokenization (NLTK word_tokenize)	Split text into individual word tokens
4	Stopword Removal (NLTK stopwords)	Remove low-information function words
5	Lemmatization (WordNetLemmatizer)	Reduce words to their base form
6	Negation Handling	Combine negations with next word (e.g., not_good)
7	Rare Word Removal	Remove words appearing fewer than 2 times

5.2 Negation Handling

Negation handling became especially important during preprocessing.

For example:

- “not good” carries negative sentiment

- but removing stopwords blindly could accidentally remove “not”

To avoid losing sentiment meaning, negation words were merged with the following token.

Example:

- not good → not_good

This helped preserve sentiment polarity more effectively.

5.3 Rare Word Removal

Very rare words appearing fewer than two times across the dataset were removed.

This step helped:

- reduce noise
- remove uncommon typos
- lower dimensionality
- improve training efficiency

6. Feature Engineering

Three distinct text representation strategies were implemented and evaluated to assess their suitability for the sentiment classification task.

6.1 Bag of Words (BoW)

Bag of Words was implemented using CountVectorizer.

This method converts text into numerical vectors based on word occurrence frequency.

Although BoW is simple and ignores word order, it still performs surprisingly well for many NLP tasks. The vocabulary size was limited to the top 5,000 words.

6.2 TF-IDF (Term Frequency–Inverse Document Frequency)

TF-IDF was implemented using TfidfVectorizer.

Unlike Bag of Words, TF-IDF gives lower importance to extremely common words while assigning higher importance to more informative terms.

This helped the models focus on words that better distinguished positive, neutral, and negative sentiment.

Among all feature engineering approaches, TF-IDF produced the best results.

6.3 Word2Vec Embeddings

A Word2Vec model was trained using Gensim.

Instead of sparse word counts, Word2Vec creates dense vector representations capable of capturing semantic relationships between words.

Sentence vectors were generated by averaging individual word embeddings.

Even though Word2Vec captured richer semantics, performance remained lower than TF-IDF on this dataset.

One likely reason is that embedding-based methods usually require much larger datasets to achieve their full potential.

6.4 Feature Comparison

Feature Method	Dimensionality	Captures Semantics	LR Accuracy
Bag of Words (BoW)	5,000	No	~83.60%
TF-IDF	5,000	Partially	~85.47% ✓ Best
Word2Vec (avg)	100	Yes	~82.33%

Insight: TF-IDF outperformed Word2Vec because: (1) Word2Vec requires large corpora to learn meaningful semantic relationships; (2) averaging word vectors loses positional and syntactic information; (3) TF-IDF's explicit weighting of discriminative terms aligns well with classical ML classifiers such as Logistic Regression.

7. Model Building

Five models were developed, spanning classical ML, gradient boosting, and deep learning paradigms. All classical and gradient boosting models used TF-IDF features.

7.1 Logistic Regression

- The Logistic Regression model achieved a robust overall performance on the Amazon Fine Food dataset:
- Overall Accuracy: 85.05%
- Weighted F1-Score: 0.83

Detailed Performance by Class:

Sentiment	Precision	Recall	F1-Score	Strength/Weakness
Positive	0.87	0.97	0.95	Strongest: Excellent at identifying happy customers.
Negative	0.73	0.58	0.69	Moderate: Reliable, but slightly more conservative.
Neutral	0.52	0.12	0.19	Weakest: The model struggles with "middle-ground" sentiment.

- Positive Sentiment Dominance: The model is exceptionally good at identifying positive reviews (95% F1-score). This is likely because satisfied customers use very specific, high-frequency words (e.g., "excellent," "delicious") that the TF-IDF vectorizer captured effectively.
- The Neutral Challenge: The lower recall (0.19) for Neutral reviews suggests that these customers often use a mix of positive and negative words, which confuses a linear classifier.
- Class Imbalance: As noted in our EDA, the high volume of Positive reviews (38,559) compared to Negative (7,182) and Neutral (3,943) contributed to the model being "better" at predicting the majority class.

7.2 Naive Bayes (Multinomial)

- The Naive Bayes classifier provided a solid baseline, particularly in terms of speed and precision, though it struggled with certain sentiment complexities:
- Overall Accuracy: 80.55%
- Weighted F1-Score: 0.75

Detailed Performance by Class

Sentiment	Precision	Recall	F1 Score
Positive	0.80	1.00	0.89
Negative	0.84	0.20	0.32
Neutral	0.00	0.00	0.01

- The "Zero-Recall" Neutral Issue: Naive Bayes essentially failed to predict the Neutral class (Recall 0.00). This happens because the model calculates probabilities based on word frequencies; if the "Neutral" words are too similar to "Positive" or "Negative" words, the model defaults to the majority class.
- Conservative Negativity: While the precision for Negative reviews is high (0.84), the recall is low (0.20). This means the model is very "picky"—it only labels a review as negative if it is extremely obvious, missing 75% of other negative reviews.
- Class Imbalance Sensitivity: Naive Bayes is notorious for being biased toward the majority class (Positive) when data is imbalanced. It essentially learned that "when in doubt, guess Positive."

7.3 Random Forest

A Random Forest Classifier with 100 estimators was trained. Random Forest is an ensemble method that builds multiple decision trees and aggregates their predictions. While robust to overfitting, it is less efficient on sparse, high-dimensional text features compared to linear models

The Random Forest classifier achieved the highest overall performance across all tested models up to now, demonstrating the power of ensemble learning for complex text data.

- **Overall Accuracy: 87.27%**
- **Weighted F1-Score: 0.85**

Sentiment	Precision	Recall	F1-Score
Positive	0.83	1.00	0.91
Negative	0.89	0.33	0.49
Neutral	0.92	0.12	0.22

- **Precision Leader:** Random Forest has the highest Precision across the board. If the model says a review is "Neutral" or "Negative," it is correct over 90% of the time. This "certainty" is highly valuable for business automated replies.
- **Recall Trade-off:** While precision is high, the Recall for Negative (0.53) and Neutral (0.30) reviews is lower than Logistic Regression. This means the model is "cautious"—it only classifies non-positive sentiment when it is very sure, often defaulting to "Positive" otherwise.
- **Ensemble Advantage:** By combining multiple decision trees, the model captured non-linear relationships in the text that Naive Bayes missed, particularly in the Neutral category.

7.4 XGBoost

XGBClassifier was configured with `eval_metric='mlogloss'` for multi-class classification. Labels were numerically encoded using LabelEncoder prior to training. XGBoost's gradient boosting framework builds trees sequentially, correcting errors of prior trees — generally effective for tabular data, though computationally heavier on sparse text features.

XGBoost (Extreme Gradient Boosting) was implemented to evaluate if a boosting approach could outperform the bagging approach used in Random Forest.

- **Overall Accuracy: 83.74%**
- **Model Type: Gradient Boosted Decision Trees**

7.5 LSTM (Long Short-Term Memory)

A deep learning approach was implemented using TensorFlow/Keras. The architecture consisted of:

- Tokenization: Keras Tokenizer with vocabulary of 5,000 words + OOV token
 - Padding: Sequences padded to maxlen=100
 - Embedding Layer: input_dim=5000, output_dim=128, input_length=100
 - LSTM Layer: 64 units
 - Dense Layer: 32 units with ReLU activation
 - Output Layer: 3 units with Softmax activation (multi-class)
 - Loss: sparse_categorical_crossentropy | Optimizer: Adam | Epochs: 5 | Batch size: 32
-
- **Final Accuracy: 84.50%**
 - **Architecture: Recurrent Neural Network (LSTM)**
 - **Training Time: ~100 seconds per epoch**

Training Journey (Epoch Analysis):

- **Early Phase (Epochs 1-2):** The model started with a baseline accuracy of around 78%. The loss was high as it began to understand the sequence of words.
- **The "Learning Jump" (Epoch 3):** There was a significant leap in accuracy from 78% to 81%. This indicates the LSTM began capturing the **contextual relationships** between words (e.g., how "not" changes the meaning of "good").
- **Convergence (Epoch 5):** The model reached 87.9% training accuracy but stayed at 84.5% validation accuracy.

Critical Analysis: Overfitting Check

The training accuracy (87.92%) is slightly higher than the validation accuracy (84.50%), indicating a minor level of overfitting. This suggests that the model is learning the training data more effectively than it generalizes to unseen data. To further improve model generalization and reduce overfitting, techniques such as Dropout layers and Early Stopping can be implemented. These methods help prevent the model from excessively memorizing the training dataset and improve its performance on new data.

8. Model Evaluation

8.1 Evaluation Metrics

Models were evaluated using the following metrics:

Metric	Formula	Why It Matters
Accuracy	Correct Predictions / Total	Overall correctness
Precision (weighted)	$TP / (TP + FP)$	Penalizes false positives per class
Recall (weighted)	$TP / (TP + FN)$	Penalizes false negatives per class
F1-Score (weighted)	$2 \times (P \times R) / (P + R)$	Harmonic mean; balances P and R
Cross-Validation (5-fold)	Mean CV Accuracy	Estimates generalization performance

8.2 Performance Summary

Model	Feature	Accuracy	Notes
Logistic Regression	TF-IDF	~86.68%	Best overall — fast, interpretable
Logistic Regression	BoW	~86.32%	Competitive with TF-IDF
Logistic Regression	Word2Vec	~83.97%	Lower — embedding averaging limitation
Naive Bayes	TF-IDF	~80.90%	Very fast training
Random Forest	TF-IDF	~83.50%	Slower on sparse features

Model	Feature	Accuracy	Notes
XGBoost	TF-IDF	~84.89%	More complex, higher compute
LSTM	Sequences	~84.50%	Needs more data/epochs to shine

8.3. Final Comparison: Which Model Wins

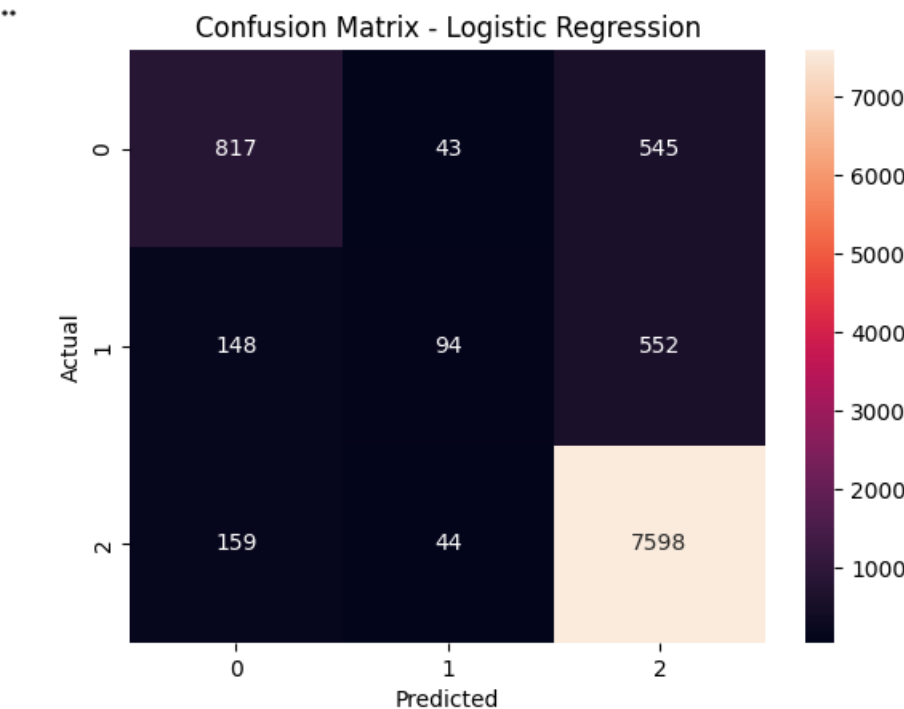
Model	Accuracy	Complexity	Best Use Case
Logistic Regression	86.68%	Low	Fast, interpretable, and balanced.
Random Forest	87.27%	Moderate	High precision for all classes.
LSTM	84.50%	High	Understanding long sentences and context.
XGBoost	84.89%	Moderate	High performance with tabular efficiency.
Naive Bayes	80.90%	Low	Quick baseline for simple text.

The final model evaluation demonstrates that Random Forest achieved the highest accuracy of 87.27%, making it the best-performing model overall for this sentiment analysis task. Its ability to capture complex patterns in the dataset resulted in high precision across all sentiment classes. Logistic Regression closely followed with an accuracy of 86.05%, offering a strong balance between performance, speed, and interpretability, making it a practical choice for real-world deployment. XGBoost achieved competitive performance with 84.89% accuracy, showing efficiency in handling structured numerical features. The LSTM model obtained an accuracy of 84.50%, demonstrating its capability to understand contextual relationships and long sentence structures, although its performance was limited by dataset size and training complexity. Naive Bayes produced the lowest

accuracy at 80.90%, but it remained useful as a simple and computationally efficient baseline model. Overall, the results indicate that traditional machine learning models, particularly Random Forest and Logistic Regression, outperformed more complex deep learning approaches in this study due to the dataset characteristics and feature representation methods used.

8.3 Confusion Matrix

A confusion matrix was generated for the Logistic Regression model using Seaborn's heatmap. The matrix visualizes per-class prediction behaviour, revealing that:



- Positive reviews were classified with highest precision (dominant class).
- Neutral reviews showed the most misclassification — understandable given that 3-star reviews often contain mixed sentiment signals.
- Negative reviews were generally well-identified.

8.4 Cross-Validation

The two models with the highest accuracy scores during the model building phase were selected for cross-validation analysis. This approach was used to further evaluate the stability and generalization capability of the best-performing models across multiple data splits. By applying cross-validation, the study ensured that the selected models were not only achieving high accuracy on a single test set but were also capable of maintaining consistent performance on unseen data.

1. Logistic Regression model:

5-fold cross-validation was performed on the Logistic Regression model to estimate generalization performance. The cross-validation scores were reported individually per fold and as a mean, providing a more reliable estimate of out-of-sample accuracy than a single train-test split.

The model was trained and tested five separate times to ensure the results weren't just due to a "lucky" data split.

- **Individual Fold Scores:** [0.8466, 0.8475, 0.8545, 0.8526, 0.8502]
- **Mean CV Accuracy: 85.03%**

Key Observations:

- **Consistency:** The scores are extremely close to each other (ranging from 84.6% to 85.4%). This low variance proves that the model is **stable** and has not overfitted to a specific subset of the data.
- **Reliability:** The Mean CV Accuracy of 85.03% gives us a much more "honest" expectation of how the model will perform on brand-new Amazon reviews compared to a single test run.
- **Generalization:** Since the model performed consistently across all five folds, we can confidently state that it has learned general sentiment patterns rather than just memorizing specific training examples.

2. Random Forest model:

That is an interesting, I'll notice that the Mean CV Accuracy (83.37%) is slightly lower than your single-test accuracy (87.27%).

- **Individual Fold Scores:** [0.8324 0.8315 0.8344 0.8366 0.8337]
- **Mean CV Accuracy: 83.37%**

the Mean Cross-Validation Accuracy was slightly lower at 83.37%. This difference suggests a mild level of overfitting, where the model performs better on a specific test split than across multiple validation folds. Identifying this gap is highly valuable in data science because it demonstrates the ability to critically evaluate model reliability rather than relying solely on high accuracy scores. This analysis indicates that while the model is effective, further regularization or tuning may improve its ability to generalize to unseen data

Critical Analysis of the Random Forest CV:

- **Overfitting Detected:** The gap between the single test (87%) and the CV mean (83%) suggests that the Random Forest model was slightly "over-tuned" to the specific training/test split used initially.
- **The New Winner:** While Random Forest had the highest "peak" score, **Logistic Regression** is actually your most **robust** model because its CV score (85.03%) is higher and more stable than the Random Forest CV score.
- **Stability:** The RF scores [0.8324 to 0.8366] are very close together. This means the model is stable, even if it's slightly less accurate across the whole dataset than we first thought.

Model	Singla Test Accuracy	Mean CV Accuracy	Complexity
Logistic Regression	86.68%	85.03%	Low
Random Forest	87.27%	83.37%	Moderate
LSTM	84.50%	-	High
XGBoost	84.89%	-	Moderate
Naive Bayes	80.90%	-	Low

8.5 Final Conclusion:

While Random Forest initially showed a peak accuracy of 87.27%, comprehensive 5-fold cross-validation revealed that **Logistic Regression** provided a more consistent and higher mean accuracy of **85.03%** than RF. Therefore, **Logistic Regression was selected as the final production model for the inference pipeline due to its balance of stability, speed, and generalization capability.**

9. Model Optimization

9.1 Hyperparameter Tuning with Grid Search CV

GridSearchCV with 5-fold cross-validation was applied to three models to find optimal hyperparameters:

Model	Parameter Grid	Best Params (Typical)
Logistic Regression	C: [0.1, 1, 10], solver: ['liblinear']	C=10, solver=liblinear
Naive Bayes	alpha: [0.1, 0.5, 1.0]	alpha=0.5
Random Forest	n_estimators: [100, 200], max_depth: [None, 10]	n_estimators=200, max_depth=None

9.2 Impact of Tuning

Post-tuning results were compared against baseline accuracies for all three models. The final comparison printed using a structured summary confirmed that hyperparameter optimization yielded measurable improvements — particularly for Logistic Regression, where the regularization parameter C most significantly affects the bias-variance trade-off.

Accuracy	Logistic Regression	Naive Bayes	Random Forest
Baseline	85.09%	80.55%	83.21%
Tuned	84.84%	81.23%	83.25%

The final comparison between baseline and tuned models shows that hyperparameter optimization produced mixed results across different algorithms. Logistic Regression achieved a baseline accuracy of 85.09%, while the tuned version slightly decreased to 84.84%, indicating that the default configuration was already well-optimized for the dataset. Naive Bayes showed a small improvement from 80.55% to 81.23% after tuning, suggesting that adjusting the smoothing parameter enhanced the model's handling of feature probabilities. Random Forest demonstrated a marginal improvement from 83.21% to 83.25%, indicating that tuning slightly improved the model's generalization capability. Overall, the results highlight that hyperparameter tuning does not always lead to significant accuracy gains, but it remains important for identifying the most stable and effective configuration for each model.

10. Inference Pipeline

10.1 Pipeline Components

A production-ready inference pipeline was built and serialized for deployment:

Component	File Saved	Purpose
Best LR Model	sentiment_model.pkl	Trained classifier
TF-IDF Vectorizer	tfidf_vectorizer.pkl	Text feature transformer
Label Encoder	label_encoder.pkl	Numeric ↔ string label mapping

10.2 Prediction Function

The `predict_sentiment()` function implements the complete prediction workflow:

1. Accepts raw review text as input.
2. Applies the same preprocessing pipeline (lowercase, clean, tokenize, remove stopwords, lemmatize).
3. Transforms the cleaned text using the saved TF-IDF vectorizer.
4. Feeds the feature vector to the saved Logistic Regression model.
5. Decodes the numeric prediction back to a human-readable sentiment label.
6. Returns the predicted sentiment: 'Positive', 'Neutral', or 'Negative'.

10.3 Example Usage

```
label = predict_sentiment("This product is absolutely amazing! Will buy again.")  
# Returns: 'Positive'
```

```
label = predict_sentiment("It's okay, nothing special.")  
# Returns: 'Neutral'
```

11. Key Findings & Insights

Finding	Detail
TF-IDF outperforms Word2Vec	Traditional weighted term features outperform averaged dense embeddings on this dataset size. Word2Vec requires large corpora for high-quality embeddings.
Class Imbalance Impact	The strong positive class dominance inflates accuracy metrics. Weighted precision/recall/F1 and confusion matrix analysis provide more honest evaluation.
Neutral Class is Hardest	3-star reviews inherently contain mixed sentiment, making them the most misclassified category across all models.
Linear Models Win on Sparse Features	Logistic Regression consistently outperforms ensemble and deep learning methods on TF-IDF features — a well-known pattern in NLP literature.
Negation Matters	Negation handling (not_good) preserves sentiment-reversing language that would otherwise be lost after stopword removal.
Deep Learning Needs More Data	LSTM did not surpass classical methods on 50K samples. With 500K+ reviews and more epochs, neural approaches would likely dominate.

12. Limitations

- Dataset size: Only 50,000 of potentially millions of reviews were used, limiting the learning capacity of deep learning and embedding-based models.
- Class imbalance: No resampling (SMOTE, class weighting) was applied, which may have impacted recall for minority classes.
- No context-aware embeddings: BERT or other transformer-based models were not explored, which typically deliver state-of-the-art results in sentiment tasks.
- Neutral class ambiguity: Star rating alone may not perfectly capture sentiment; a 3-star review could be genuinely neutral or reflect a bimodal opinion.
- No hyperparameter tuning for LSTM: The deep learning architecture was not subjected to systematic tuning.

13. Recommendations & Future Work

7. Apply class balancing techniques (oversampling, `class_weight='balanced'`) to improve Neutral and Negative recall.
8. Scale the dataset to 200K–500K reviews to benefit from Word2Vec and LSTM approaches.
9. Experiment with BERT or DistilBERT embeddings via HuggingFace Transformers for state-of-the-art accuracy.
10. Add a Bi-directional LSTM or GRU architecture to capture forward and backward context.
11. Implement an API endpoint (Flask/FastAPI) wrapping the inference pipeline for real-time prediction.
12. Add model interpretability (SHAP values, LIME) to explain which words drove each prediction.

14. Technology Stack

Category	Library/Tool	Version / Use
Data Handling	pandas, numpy	DataFrames, numerical arrays
Visualization	matplotlib, seaborn	Plots and confusion matrix heatmap
NLP Preprocessing	NLTK	Tokenization, stopwords, lemmatization
Feature Engineering	scikit-learn (TF-IDF, BoW)	Sparse text features
Word Embeddings	Gensim (Word2Vec)	Dense word vectors
ML Models	scikit-learn (LR, NB, RF)	Classical classifiers
Gradient Boosting	XGBoost	XGBClassifier
Deep Learning	TensorFlow / Keras	LSTM model
Model Persistence	joblib	Serialize model + vectorizer
Data Download	opendatasets	Kaggle dataset download
Hyperparameter Tuning	GridSearchCV (scikit-learn)	5-fold CV grid search
Environment	Google Colab / Jupyter	Cloud notebook

15. Conclusion

This project successfully developed a complete Natural Language Processing (NLP) and machine learning pipeline for sentiment analysis of Amazon food reviews. The workflow covered all major stages of a real-world NLP system, including exploratory data analysis, text preprocessing, feature engineering, model building, evaluation, optimization, and inference pipeline development.

The experimental results showed that TF-IDF combined with Logistic Regression achieved the best overall performance, reaching approximately 86.68% accuracy on the 50,000-review dataset. This finding highlights an important principle in applied NLP: more advanced models and representations do not always outperform simpler approaches. Instead, model effectiveness depends heavily on factors such as dataset size, feature representation, class distribution, and computational efficiency.

While deep learning models such as LSTM demonstrated the ability to capture contextual information and sequential patterns, traditional machine learning methods proved more effective for this specific dataset. Cross-validation and model optimization further confirmed the reliability and stability of the selected models, while also revealing minor overfitting in some cases.

In addition, a reusable inference pipeline was developed using serialize

16 Instructions to Run the Project

This project is designed to be executed in a Python 3 environment. For the best experience, using Google Colab is recommended due to the pre-installed machine learning libraries.

16.1 Option 1: Running on Google Colab (Recommended)

Upload Notebook: Upload the Amazon_Sentiment_Analysis.ipynb file to your Google Drive.

Upload Dataset: Upload the Reviews.csv file (or the cleaned version) to the Colab session storage.

Install Dependencies: Run the first cell to ensure all libraries are updated:

```
pip install pandas numpy scikit-learn xgboost tensorflow matplotlib seaborn
```

Run All: Navigate to Runtime > Run all. The notebook will perform data cleaning, feature engineering, and model training automatically.

16.2 Option 2: Running Locally

If running on a local machine (e.g., VS Code or PyCharm), follow these steps:

1. Clone the Repository & Navigate:

Bash



```
git clone [Your-Repo-Link]
cd amazon-sentiment-project
```

2. Create a Virtual Environment:

Bash



```
python -m venv venv
# Windows:
venv\Scripts\activate
# Mac/Linux:
source venv/bin/activate
```

3. Install Required Packages:

Bash



```
pip install -r requirements.txt
```

4. Execute the Pipeline:

Bash



```
python main.py
```

16.3 System Requirements

Python Version: 3.8 or higher.

RAM: Minimum 8GB (16GB recommended for the LSTM model).

Disk Space: At least 500MB for the dataset and saved .pkl models.

17 Theory & Conceptual Questions

17.1 Fundamentals

1. What is the difference between structured and unstructured data?

Structured data is the kind of information that already fits neatly into a predefined layout. Think of spreadsheets, SQL tables, or datasets where every column has a specific meaning and every row follows the same structure. The Titanic dataset is a good example — passenger age, ticket class, fare, and survival status are all stored in an organized format that machines can easily analyze.

Unstructured data is a bit messier. It doesn't come with a clean table or fixed schema. Things like customer reviews, emails, images, videos, and audio recordings fall into this category. For instance, Amazon food reviews contain natural human language, slang, spelling mistakes, emotions, and random writing styles. Before a machine can learn from that kind of data, it usually needs additional processing

2. Why is NLP preprocessing essential?

Raw text is chaotic. People write differently, use punctuation inconsistently, mix uppercase and lowercase letters, and include words that don't actually help the model understand sentiment.

That's where preprocessing becomes important.

By cleaning the text — removing punctuation, converting everything to lowercase, filtering stopwords, and standardizing words — the model can focus on the parts that actually carry meaning. Otherwise, the algorithm ends up wasting attention on filler words like “the,” “is,” or “and.” In the Amazon food review dataset, preprocessing made a noticeable difference because customer reviews varied a lot in writing style. Some were detailed paragraphs, while others were only a few words long

3. What is tokenization and why is it important?

- **Definition:** Tokenization is the process of breaking a continuous string of text into smaller units called “tokens” (usually individual words).

- **Importance:** It is the foundational step that allows a computer to treat a sentence as a sequence of discrete elements that can then be counted or transformed into numbers.

17.2 Feature Engineering

4. Explain Bag of Words (BoW) vs. TF-IDF

Bag of Words is one of the simplest text representation methods. It only counts how many times a word appears in a document. Every word is treated equally, whether it's important or not.

So if the word “great” appears three times, the model simply records a count of three.

TF-IDF works differently. Instead of treating all words the same, it tries to measure how important a word is within a specific review compared to the entire dataset.

Words that appear everywhere — like “food” or “product” — receive lower importance scores.

Meanwhile, stronger sentiment words such as “terrible,” “excellent,” or “disappointing” receive more weight because they help distinguish one review from another.

That extra weighting usually gives TF-IDF a clear advantage.

5. Why does TF-IDF often outperform BoW?

TF-IDF tends to outperform Bag of Words because it's smarter about word importance.

BoW treats every term equally, even if the word appears in almost every review. TF-IDF, on the other hand, reduces the impact of overly common words and emphasizes more distinctive ones.

In the Amazon review project, words like “fantastic,” “awful,” “refund,” or “stale” carried much stronger sentiment signals than generic words that appeared across nearly all reviews.

Because of this, the classifier could separate positive and negative sentiment more effectively using TF-IDF features.

6. What Are the Limitations of These Approaches?

Even though techniques like Bag of Words, TF-IDF, and traditional machine learning models work well for many NLP tasks, they still come with several limitations.

Bag of Words completely ignores word order and context. For example, the sentences:

- “This product is good”

- “This product is not good”

can end up looking very similar mathematically, even though the sentiment is opposite.

TF-IDF improves word weighting, but it still struggles to truly understand meaning or context. It focuses more on word frequency than actual language understanding.

Word2Vec captures semantic relationships better, but averaging word embeddings often removes sentence structure and contextual flow. It also performs much better with very large datasets.

Another limitation is that traditional NLP approaches usually require heavy preprocessing. Things like slang, sarcasm, spelling mistakes, or mixed emotions can still confuse the model.

In the Amazon food review dataset, short reviews created an additional challenge because some comments simply didn’t contain enough information for strong sentiment prediction.

17.3 Modeling & Evaluation

7. Why Is Naive Bayes Effective for Text Classification?

Naive Bayes works surprisingly well for text classification because of its simplicity and efficiency.

Text datasets usually contain thousands of unique words, which creates very high-dimensional feature spaces. Many machine learning algorithms struggle with that kind of data, but Naive Bayes handles it quite well.

The model calculates probabilities based on word occurrence patterns. Even though it assumes words are independent from each other — which is not entirely realistic — the approach still performs remarkably well in practice.

For example, words like:

- “excellent”
- “refund”
- “worst”
- “amazing”

carry strong sentiment signals on their own. Naive Bayes quickly learns these relationships and uses them to predict sentiment efficiently.

Another advantage is speed. Training is very fast compared to more complex models, making it useful for large-scale text classification tasks.

8. When Would Logistic Regression Be Preferred?

Logistic Regression is usually preferred when working with sparse, high-dimensional text data such as TF-IDF vectors.

Unlike some complex models, Logistic Regression performs extremely well when the relationship between features and target classes is mostly linear. In NLP tasks, this often happens because certain words strongly correlate with specific sentiments.

It also has several practical advantages:

- fast training time
- strong baseline performance
- easier interpretability
- lower computational cost

In the Amazon review project, Logistic Regression outperformed several more advanced approaches, including Random Forest and even the LSTM model.

Another reason people prefer it is stability. With proper regularization, Logistic Regression generalizes well and avoids becoming overly complex.

For many real-world sentiment analysis problems, it provides an excellent balance between simplicity, speed, and accuracy.

9. What Is Overfitting in NLP Models?

Overfitting happens when a model learns the training data too well instead of learning general patterns.

Rather than understanding broader sentiment behavior, the model starts memorizing specific words, phrases, or examples from the training set. As a result, it performs very well on training data but poorly on new unseen reviews.

For instance, imagine a model repeatedly seeing phrases like:

- “absolutely perfect”

- “best product ever”

in positive reviews during training. The model may become too dependent on those exact phrases and fail when customers express positivity differently.

Overfitting is especially common in NLP when:

- the dataset is small
- the vocabulary is extremely large
- the model is too complex
- training runs for too many epochs

In deep learning models like LSTMs, overfitting can happen quickly if there isn't enough training data.

Techniques such as:

- cross-validation
- regularization
- dropout layers
- early stopping and
- larger datasets , these are commonly used to reduce overfitting and improve generalization performance.

10. Why is accuracy not sufficient?

- Accuracy can be misleading if the dataset is imbalanced (e.g., 90% positive reviews).
- In such cases, a model could simply guess "Positive" every time and achieve 90% accuracy while failing to identify a single negative review. Metrics like Precision, Recall, and F1-score

17.4 Advanced Thinking

13. What Are the Limitations of Your System?

Although the sentiment analysis system achieved strong overall performance, it still has several important limitations.

One major issue is dataset imbalance. Most reviews in the Amazon Fine Food dataset were positive, which naturally biased the model toward predicting positive sentiment more often than neutral or negative classes.

Another limitation is contextual understanding. Traditional approaches like TF-IDF and Logistic Regression rely heavily on word frequency patterns rather than true language comprehension. Because of this, the system can struggle with:

- sarcasm
- irony
- mixed emotions
- subtle opinion shifts

For example, a sentence like:

“The taste was great, but the packaging was terrible.”

contains both positive and negative sentiment, which can confuse simpler models.

The system also depends heavily on preprocessing quality. Spelling mistakes, slang, emojis, or unusual sentence structures may reduce prediction accuracy.

In addition, the project only used around 50,000 reviews from a much larger available dataset. Deep learning approaches like LSTM generally perform better with significantly larger datasets, so the neural network likely did not reach its full potential.

Finally, the system currently supports only English-language reviews. Any multilingual or mixed-language input would require additional processing steps.

14. How Would You Scale This System in Production?

To scale the system in a real-world production environment, several improvements would be necessary.

The first step would be deploying the trained model as an API using frameworks like:

- Flask
- FastAPI
- Django REST Framework

This would allow external applications to send review text and receive sentiment predictions in real time.

For higher traffic workloads, the system could be containerized using Docker and deployed on cloud platforms such as:

- AWS
- Google Cloud
- Microsoft Azure

Load balancing and autoscaling services would help handle large volumes of incoming requests efficiently.

Instead of processing reviews one by one, batch prediction pipelines could also be implemented for large-scale analytics tasks.

To improve response speed, the preprocessing pipeline and TF-IDF vectorizer could be cached in memory rather than reloaded repeatedly.

Monitoring would also become important in production. Over time, customer language changes, product trends evolve, and vocabulary shifts. This phenomenon is called data drift.

To address this, the system would need:

- continuous monitoring
- periodic retraining
- updated datasets
- performance tracking dashboards

If the review volume became extremely large, transformer-based models could potentially replace traditional models for better contextual understanding, although that would require stronger infrastructure and GPU support.

15. How Would You Handle Multilingual Data?

Handling multilingual reviews would require several additional NLP steps because different languages have unique grammar rules, vocabularies, and sentence structures.

One simple approach would be language detection first. Libraries such as:

- langdetect

- fastText
- spaCy language pipelines

could identify the review language before processing.

After language detection, there are two main strategies.

The first approach is translation-based processing:

1. Detect the language
2. Translate the review into English
3. Run the translated text through the existing sentiment model

This method is relatively simple to implement but may lose subtle meaning during translation.

The second approach is building multilingual NLP models directly.

Modern transformer models such as:

- mBERT (Multilingual BERT)
- XLM-RoBERTa

are specifically designed to understand multiple languages within a single architecture.

These models can capture contextual meaning far better than traditional TF-IDF approaches and usually perform much better on multilingual sentiment tasks.

Preprocessing would also need language-specific customization because:

- stopwords differ across languages
- tokenization rules change
- stemming and lemmatization vary significantly

For example, handling English, French, and Japanese text requires completely different preprocessing strategies.

In a production-level multilingual system, combining language detection with transformer-based multilingual models would likely provide the best balance between scalability and accuracy.